

Database System Assignment 1

Q1i) Cyber Nova faces several issues due to file-based systems.

- **Data Redundancy**: Same customer or service information is stored in multiple files at different branches, leading to unnecessary duplications.
- **Inconsistent Data**: Updates in one branch may not reflect in another causing conflicting or outdated records.
- **Limited Data Sharing**: Branches can't easily share data with each other as files are isolated.
- **Poor Data Security**: File based systems typically have minimal access control, making sensitive data vulnerable.
- **Slow Data Retrieval**: Searching, sorting, or generating reports from multiple files is time-consuming.
- **Difficulty in Maintaining Integrity**: Ensuring correctness (like unique customer ID's or valid incident records) is hard without centralized control.
- **Scalability Issues**: As the company grows, managing files across multiple branches becomes inefficient and error-prone.

ii) A DBMS provides structured, centralized, and efficient data management. Benefits include

- **Data Centralization**: Stores all customer, incident, and service records in a single database, reducing redundancy.
- **Data Integrity**: Enforces rules / constraints (eg: primary keys, foreign keys, unique constraint) to maintain consistent accurate data.
- **Efficient Data Retrieval**: Supports query language (like SQL) for fast searches, reports, and analytics.

- **Data Security**: Provides controlled access via user roles, ensuring only authorized personnel can view or modify data.
- **Concurrent Access**: Multiple users from different branches can easily access and update data simultaneously without conflicts.
- **Backup and Recovery**: Automatic mechanisms protect data from accidental loss.
- **Scalability and Flexibility**: Can handle increasing data volume and evolving requirements easily.

iii) A server based mobile app would act as a bridge between employees and the central database.

- **Remote Access**: Employees at any branch can access customer, incident, or service data via their mobile devices.
- **Real-time Updates**: Changes made by one employee are immediately available to others across branches.
- **Improved Productivity**: Employees can enter or retrieve data from anywhere, reducing the need to visit the main office.
- **Enhanced Collaboration**: Branches can co-ordinate more effectively with a shared system.
- **User Friendly Interface**: Simplifies interaction with the database, making it easier for non-technical staff to use.

Q2i) SELECT * FROM Clients
WHERE country = 'Pakistan';

ii) SELECT service-name, service-cost FROM Services
WHERE service-cost > 500;

iii) SELECT * FROM Service-Records
WHERE status = 'Completed';

iv) SELECT client-name FROM Clients
WHERE country IN ('Pakistan', 'United States');

v) SELECT * FROM Services
WHERE Service-cost BETWEEN 300 AND 1000;

vi) SELECT DISTINCT location FROM Offices;

vii) SELECT * FROM Service-Records
ORDER BY service-data DESC;

viii) SELECT UPPER(client-name) AS client-name-uppercase
FROM Clients;

ix) SELECT COUNT(*) AS total-service-records
FROM Service-Records;

x) SELECT service-name, service-cost FROM Services
ORDER BY service-cost ASC;

Q3i) INSERT INTO Borrow-Records (Record-ID, Member-ID, Book-ID, Borrow-Date, Return-Date)
VALUES (1004, 104, 1, '2026-02-05', NULL);

Result: This will fail as Member-ID 104 doesn't exist in the members table. (Foreign key violated)

ii) UPDATE Books
SET Copies-Available = -1
WHERE Book-ID = 2;

Result:- This will fail cause of the CHECK constraint (copies-Available > 0).

iii) DELETE FROM Members
WHERE Member-ID = 101;

iv) INSERT INTO Members (Member-ID, Member-Name, Email, Membership-Type)
VALUES (104, NULL, 'faizan@email.com', 'Premium');

Result: This will fail because Member-Name has NOT NULL constraint.

v) UPDATE Borrow-Records
SET Book-ID = 5
WHERE Record-ID = 1002;

Result: This will fail due to foreign key constraint as Book-ID 5 doesn't exist.

vi) INSERT INTO Books (Book-ID, Title, Author, Copies-Available)
VALUES (4, 'Network Security', 'Ali Raza', 2);

vii) UPDATE Borrow-Records
SET Member-ID = 102
WHERE Record-ID = 1003;

viii) DELETE FROM Books
WHERE Book-ID = 3;

Result: This will fail as Book-ID 3 is referenced in Borrow-Records.

ix) INSERT INTO Borrow-Records (Record-ID, Member-ID, Book-ID,
Borrow-Date, Return-Date)
VALUES (1005, 102, 3, '2026-02-07', '2026-02-15');

x) ALTER TABLE Books
ADD CONSTRAINT chk-copies-available
CHECK (Copies-Available $\geq$ 1);

Q41) There are 7 tables in the schema

- BOOK
- BOOK-COPIES
- BORROWER
- BOOK-AUTHORS
- BOOK-LOANS
- PUBLISHER
- LIBRARY-BRANCH

ii) Table Name	Primary Key	
BOOK	BookID	
PUBLISHER	Name	
BOOK-AUTHORS	(BookID, Author Name)	{ composite Key
BOOK-COPIES	(BookID, BranchID)	{ composite Key
LIBRARY-BRANCH	BranchID	
BORROWER	Card No	
BOOK-LOANS	(BookID, BranchID, Card No)	{ composite key

iii) Parent Table (Referenced)	Child Table (Referencee)	Foreign Key (in child table)	Purpose
PUBLISHER (Name)	BOOK	Publisher Name	Ensures every book is linked to a valid publisher
BOOK (BookID)	BOOK-AUTHORS		
BOOK (BookID)	BOOK-AUTHORS	BookID	Ensures authors are associated with existing books only.
BOOK (BookID)	BOOK-COPIES	BookID	Ensures copies are recorded only for valid books.

Date: _____

LIBRARY-BRANCH (BranchID)	BOOK-COPIES	BranchID	Ensures copies belong to valid branch.
BOOK(BookID)	BOOK-LOANS	BookID	Ensures loans are for existing books
LIBRARY-BRANCH (BranchID)	BOOK-LOANS	BranchID	Ensures loans are issued from a valid branch
BORROWER(CardNo)	BOOK-LOANS	CardNo	Ensures loans are linked to valid borrowers

iv) Table Name	ATTRIBUTE	Domain
BOOK	BookID	INT, PRIMARY KEY NOT NULL
	Title	VARCHAR(100), NOT NULL
	Publisher Name	VARCHAR(100), FOREIGN KEY (^{Publisher} Name) references PUBLISHER(Name), NOT NULL
BOOK-AUTHORS	BookID	INT, FOREIGN KEY (BookID)
		references BOOK(BookID) NOT NULL
	AuthorName	VARCHAR(100) NOT NULL CONSTRAINT pk PRIMARY KEY (BookID, AuthorName)
PUBLISHER	Name	VARCHAR(100) PRIMARY KEY NOT NULL
	Address	VARCHAR(200) Phone
	Phone	VARCHAR(15)
Page		☐ Victory

Date: _____

LIBRARY-BRANCH	BranchID	INT PRIMARY KEY NOT NULL
	BranchName	VARCHAR (100) NOT NULL
	Address	VARCHAR (200) NOT NULL
BORROWER	Card No	INT PRIMARY KEY NOT NULL
	Name	VARCHAR (100) NOT NULL
	Address	VARCHAR (200)
	Phone	VARCHAR (15)
BOOK-COPIES	BookID	INT FOREIGN KEY (BookID) references BOOK (BookID) NOT NULL
	BranchID	INT FOREIGN KEY (BranchID) references LIBRARY-BRANCH (BranchID) NOT NULL
	No-of-Copies	INT CHECK (No-of-Copies > 0) NOT NULL
		CONSTRAINT pk PRIMARY KEY (BookID, BranchID)
BOOK-LOANS	BookID	INT Foreign KEY (BookID) references BOOK (BookID) NOT NULL
	BranchID	INT FOREIGN KEY (BranchID) references LIBRARY-BRANCH (BranchID) NOT NULL
	Card No	INT FOREIGN KEY (Card No) references BORROWER (Card No) NOT NULL
	Date Out	DATE NOT NULL
	Due Date	DATE
		CONSTRAINT pk PRIMARY KEY (BookID, BranchID, Card No)

iv) Generally relationship b/w book and book-authors can be many to many as a book can have multiple authors (eg course books) as well as an author can write multiple books (eg: lord of the ring series)

But as per question AN author who has written multiple books indicate one author writing multiple books so this can be a 1 to many relation. Schema can look like this

	Book ID	Book Author
Q5)	1	Tolkien
	3	Tolkien
	4	Tolkien

Q5i) Data Independence refers to the ability of a Database Management System (DBMS) to allow changes in the database structure without affecting the application programs that access the data. It separates the logical design of the database from the physical storage details. There are two main types:

- Physical Data Independence: Changes in the physical storage (such as modifying file organization, indexing, or storage devices) do not affect application programs.
- Logical Data Independence: Changes in the logical structure (such as adding a new column to a table) do not significantly effect application programs.

Example:- Suppose a company stores customer records in a database. Initially, the data is stored without indexing. Later, the database administrator adds an index to improve search performance. Even though the storage method changed internally, the application program used to

retrieve customer information continues to work without modification. This demonstrates data independence.

ii) A simple example is a small home based business that keeps daily expense records in a single Excel file. In this case, a file-based system may be preferred because:

- the amount of data is very small.
- only one person accesses and manages the data
- there is no need for multi-user access or complex security.
- It's inexpensive and easy to maintain
- No technical expertise required to manage it.

Using a full DBMS in such a small scale scenario would increase cost and complexity unnecessarily. Therefore for very simple applications, a file based system can be more practical.

iii) The difference between DELETE, TRUNCATE and DROP in SQL is,

DELETE	TRUNCATE	DROP
• Removes specific rows from a table	• Removes all rows from a table	• Completely removes the entire table
• Can be used with a WHERE clause	• Does not allow a WHERE clause	• Does not allow a WHERE clause
• The table structure remains intact	• The table structure remains intact	• Deletes both table structure and all its data
• Can usually be rolled back before committing	• Generally can't be rolled back	• Can not be rolled back
	• Faster than DELETE as it removes data in bulk	• The table must be recreated if needed again

In summary, DELETE removes selected data (eg: deleting a specific customer record), TRUNCATE removes all data quickly (eg: clearing all records from a temporary table) and DROP removes the entire table permanently. (eg removing a table that is no longer required).

iv) Referential Integrity ensures that relationships between tables remain accurate and consistent. It is enforced through foreign key constraints.

It ensures that:

- A foreign key value must match an existing primary key value in the parent table.
- A parent record can't be deleted if it is referenced by a child record (unless cascading or null is defined).

If referential integrity is not enforced, several problems may occur.

- Orphan records (records that reference non-existent data)
- Inconsistent and unreliable data.
- Errors in reports and queries.
- Loss of trust in database system.

For example, if a borrow record refers to a Book-ID that doesn't exist in the Books table, the database would contain incorrect information. Referential integrity prevents such situations.

v) Yes, a foreign key can have a NULL value, provided that it is not defined with a NOT NULL constraint.

A NULL foreign key indicates that the relationship is optional. It means that the record doesn't currently reference any parent record.

For eg: In a company database, an employee may have a Manager-ID as a foreign key. If the employee has not yet been assigned a manager, the Manager-ID can be NULL.

Having a NULL foreign key does not violate referential integrity because NULL represents the absence of a relationship, not an invalid relationship.

Referential integrity is only violated when a foreign key contains a value that does not exist in the referenced table.